BESCHREIBUNG

Rechnerbeschreibung

Aufbau, Register, Maschinenwort, Zahlendarstellung, Magnettrommel und Befehlszyklus des Cellatron C 8205

Dieses Dokument beschreibt den Rechner selbst: wie er aufgebaut ist, wie ein Maschinenwort und ein Befehl aussehen, wie die Magnettrommel organisiert ist und wie ein Befehl in vier Phasen abgearbeitet wird. Es richtet sich an Leser, die das Projekt nicht im Detail verfolgt haben, und ersetzt keines der Fachdokumente, sondern führt in sie ein.

Quellen	C8205-Interpreter-Überblick_und_Gesamtaufbau.md	0.95-001
	C8205-Interpreter-Datenstruktur_und_Verarbeitungseinheiten.md	0.95-001
	C8205-Interpreter-Magnettrommel.md	0.95-001
	C8205-Interpreter-Befehlsliste.md	0.95-001
	C8205-Interpreter-Eingänge_der_Knoten.md	0.95-001
Stand	20.10.2026	
Umfang	17 Seiten, A4 hoch	
Erzeugt mit	erzeuge_rechnerbeschreibung_pdf.py	

> Maßgeblich bleiben die Quelldateien. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Versionen oben mit den Versionszeilen der Quelldateien übereinstimmen.

Lesehinweise

Schreibweisen: Zahlen sind oktal, sofern nicht „dezimal“ vermerkt ist. ⟨x⟩ bedeutet „Inhalt von x“, z_0 … z_{32} sind die Bitstellen eines Maschinenwortes (z_0 links). Weitere Schreibweisen: Kapitel 3.

Abgrenzung: Angaben zum Bedienpult stehen nur, soweit sie zur Einordnung nötig sind. Die Annahmen für die interpretative Abarbeitung (Kapitel 9) sind mit c8205_interp.py abgeglichen (Stand des Abgleichs: 08.10.2026).

> **Achtung** Alle Zeitangaben dieses Dokuments (Wortzeit, Wartezeit, Zugriffszeit) beschreiben die Hardware. Der Python-Interpreter rechnet dagegen mit Zeit 0 (Kapitel 9).

Verwandte PDFs: Bedienpult und Peripherie · Einzelbefehlsliste · Kodetabellen · Knotengraph · Bedienungsanleitung des Interpreters · Testhandbuch · BIS.

Legende dieses Dokuments

Zeichen	Bedeutung
Hinweis (blauer Balken)	Ergänzung, die das Verständnis erleichtert.
Achtung (orangefarbener Balken)	Stelle, an der man sich leicht irrt oder an der Hardware und Interpreter abweichen.
Grafiken	Schematisch, nicht maßstäblich; sie zeigen die Zusammenhänge, nicht das Aussehen des Geräts.

Inhaltsverzeichnis

1 Einordnung und Leistungsdaten . . . 3
2 Geräteaufbau . . . 3
3 Zahlen und Schreibweisen . . . 5
3.1 Dualzahlen, ganze Zahlen und Gleitkommazahlen . . . 5
3.2 Externe Darstellung und Tetraden (Konvention des BIS) . . . 6
4 Maschinenwort und Befehlswort . . . 6
5 Register und Flags . . . 7
5.1 Register . . . 7
5.2 Flags . . . 8
5.3 Anzeige des Arbeitszustands . . . 8
6 Die Magnettrommel . . . 9
6.1 Logische und physische Reihenfolge der Sektoren . . . 9
6.2 Zeitoptimale Programmierung . . . 10
6.3 Bahnsperrenschalter . . . 11
6.4 Nachbildung im Interpreter: die persistente Trommeldatei . . . 11
7 Der Befehlszyklus: vier Phasen . . . 12
7.1 Informationszustand nach den Phasen . . . 13
7.2 Besonderheiten im Phasenablauf . . . 13
8 Befehlsarten im Überblick . . . 14
8.1 Befehlstyp B . . . 14
8.2 Operationsart A bei Organisations- und Einzelbefehlen . . . 14
8.3 Operationszusatz Z . . . 15
8.4 Abbruch bei Gruppen- und Wiederholungsbefehlen . . . 15
9 Hardware und interpretative Abarbeitung . . . 16
10 Wo steht was? – Verweise . . . 17

1 Einordnung und Leistungsdaten

Der C 8205 ist ein Einadreßautomat: Jeder Befehl nennt genau eine Speicheradresse, die zweite Größe jeder Operation steht im Akkumulator. Er wurde 1968 in der DDR gebaut. Bei zeitoptimaler Programmierung führt er etwa 2000 verdrahtete Operationen je Sekunde aus.

Als Vorzüge werden genannt:

- geringer technischer Aufwand der Zentraleinheit (nur ca. 400 Transistoren und 2000 Dioden),
- die Magnettrommel als vergleichsweise billiges Speichermedium mit relativ kleiner Zugriffszeit,
- für einen Kleinrechner hohe Operationsgeschwindigkeit,
- Programmierhilfen in Form sogenannter Interpretersysteme, die den Programmieraufwand wesentlich verringern.

Eingesetzt wird er für wissenschaftlich-technische und mathematisch-ökonomische Berechnungen, zur Datenverarbeitung in begrenztem Umfang und als Hilfs- oder Satellitenrechner im Rahmen eines Systems von Rechnern.

> **Hinweis** Der Begriff „Interpretersystem“ in dieser historischen Aufzählung meint Programmiersysteme auf dem C 8205 selbst. Er ist nicht zu verwechseln mit dem Python-Interpreter dieses Projekts, der den Rechner nachbildet.

Kenngröße	Wert
Stromversorgung	220 V ± 10 %, 50 ± 1 Hz, rund 1500 VA
Zentraleinheit	Rechenwerk und Leitwerk; ca. 400 Transistoren, 2000 Dioden
Wortlänge	33 Bit (Maschinenwort)
Speicher	Magnettrommel, 4096 Maschinenworte (128 Spuren × 32 Sektoren)
Adressierung	12 Bit Adreßteil → 4096 Zellen; 9 Bit Operationsteil → 512 Bitkombinationen
Befehlssatz	512 Codes, davon etwa 370 mit voneinander verschiedener Wirkung (Einzelbefehlsliste: 372), zusammengesetzt aus rund 20 Grundoperationen
Operationsgeschwindigkeit	Einzelbefehle min. 0,5 ms (≈ 1900 Op/s) · Organisationsbefehle min. 0,4 ms (≈ 2400 Op/s) · Gruppen-/Wiederholungsbefehle bis ca. 9000 Op/s

2 Geräteaufbau

Der Rechner besteht aus der Stromversorgungseinheit und der Zentraleinheit (Rechenwerk und Leitwerk). Zur unmittelbar zugeordneten (1.) Peripherie gehören der Magnettrommelspeicher, das Bedienpult mit Tastatur, Lampenfeld und Bahnsperrenschaltern, zwei Lochstreifenleser, ein Lochstreifenstanzer und ein Schreibwerk für alphanumerische Ein- und Ausgabe.

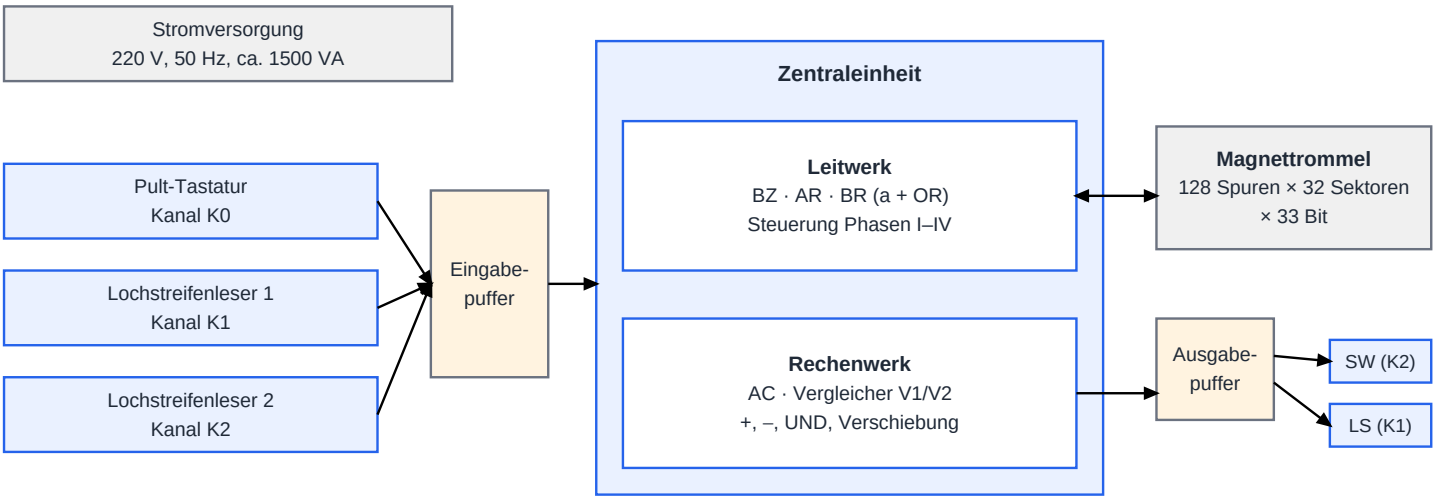


Abbildung 1: Informationsfluss zwischen den Baueinheiten. Tastatur und beide Leser speisen über einen Puffer die Zentraleinheit; Ausgaben laufen über einen weiteren Puffer zu Schreibwerk (SW) und Lochstreifenstanzer (LS).

Gerät	Kanal	Richtung	Geschwindigkeit / Besonderheit
Pult-Tastatur bzw. Schreibwerk-Eingabe	K0	Eingabe	Zeicheneingabe von Hand; Schalter T/SW wählt Tastatur oder Schreibwerk
Lochstreifenleser 1	K1	Eingabe	130 Zeichen/s (Standardleser, mechanisch mit Metallbürsten); 8-Kanal- oder 5-Kanal-Kode
Lochstreifenstanzer	K1	Ausgabe	max. 50 Zeichen/s; 8-Kanal-Kode mit Paritätslochung
Lochstreifenleser 2	K2	Eingabe	wie Leser 1
Schreibwerk	K2	Ausgabe	max. 10 Zeichen/s; Zeichenkode R 300
(frei)	K3	—	beim C 8205 ist kein Gerät zugeordnet

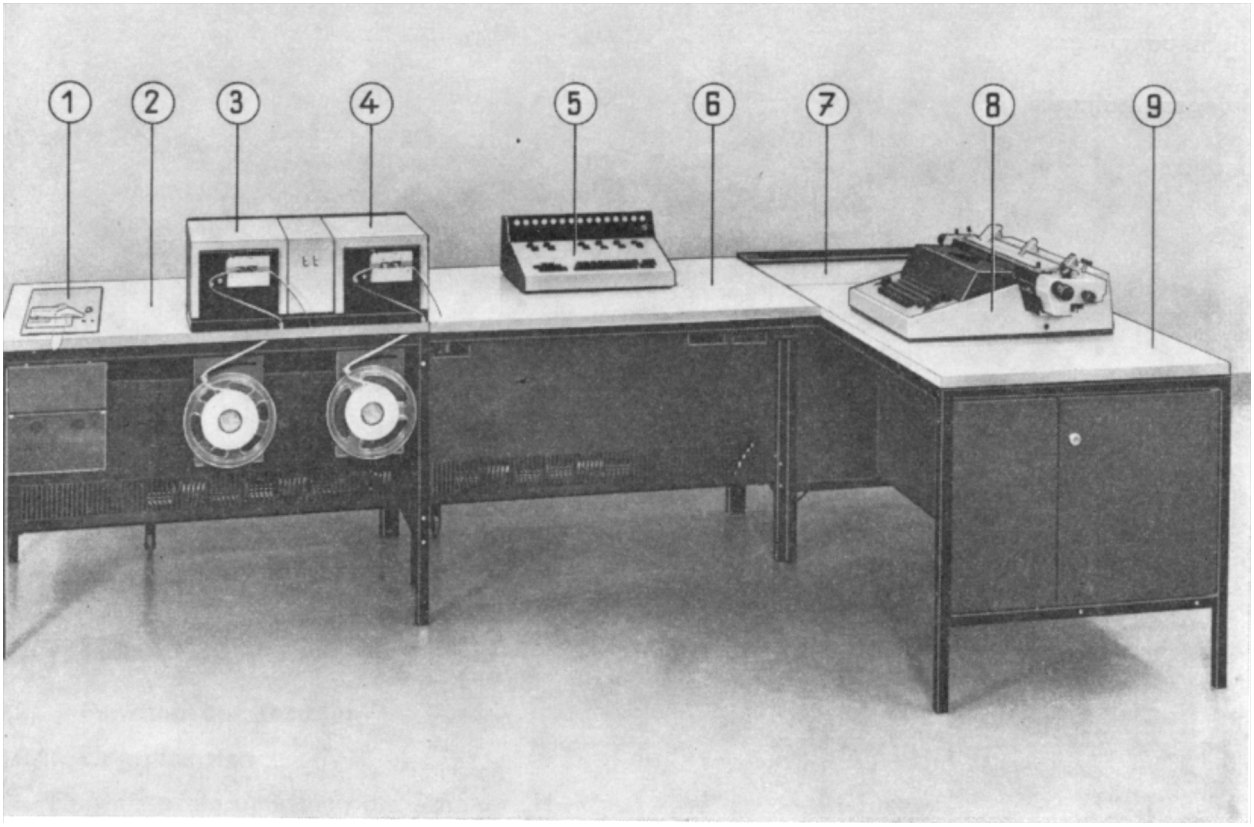


Abbildung 2: Gesamtansicht des C 8205 (Foto, Originalunterlagen). 1 Lochbandstanzer, 2 Peripherietisch, 3 Leser 2, 4 Leser 1, 5 Tastatur-Anzeige-Einheit (Bedienpult), 6 Rechentisch, 7 SVE-Tisch (Stromversorgungstisch), 8 Ein- und Ausgabe-Schreibwerk, 9 Schreibwerktisch.

Als 2. Peripherie (wechselnd anschließbar) kommen lochbanderzeugende und lochbandverarbeitende Geräte in einem beliebigen 5-Kanal- oder 8-Kanal-Kode in Frage; beim 8-Kanal-Kode liegt die Paritätslochung im 5. Kanal. Programmsysteme werden meist im 8-Kanal-R300-Kode oder im 5-Kanal-CCITT-2-Kode (Telegraphenalphabet Nr. 2) geliefert. Die Zeichenkodes stehen im PDF „Kodetabellen“, Bedienelemente und Geräteverhalten im PDF „Bedienpult und Peripherie“.

3 Zahlen und Schreibweisen

Thema	Festlegung
Zahlensystem	Zahlen in Befehlen, Registern und im Speicher sind oktal, außer wenn ausdrücklich „dual“ oder „dezimal“ gesagt wird. Bitbreiten, Nummern von Elementen und Zeitangaben in diesem Dokument sind dezimal.
Inhalt von	Spitze Klammern bedeuten „Inhalt von“: bei Registern die einzige Lesart; beim Speicher ist 1000 die Adresse 1000, ⟨1000⟩ der Inhalt dieser Zelle. Sie gelten an Stelle einer Qualifizierung, da sie nur bei Registern und Speicherinhalten vorkommen.
Bitstellen	Die 33 Bit eines Maschinenwortes heißen von links nach rechts z_0, z_1, …, z_{32} (2^{33} mögliche Wörter). Je drei Bit bilden eine Triade, also eine Oktalziffer: 11 Triaden t_{10} bis t_0.
Zahlendarstellung	Dualzahlen, ganze Zahlen und Gleitkommazahlen: Abschnitt 3.1; externe Darstellung und Tetraden (Konvention des BIS): Abschnitt 3.2.
Namensbereiche	Flags, Register, Felder des Befehlswortes sowie Knoten und Konnektoren des Knotengraphen sind getrennte Bereiche; gleiche Namen sind zulässig (Flag A / Feld A, Feld B / Konnektor B). Außerhalb des eigenen Bereichs wird qualifiziert: „Flag A“, „Feld A“.

3.1 Dualzahlen, ganze Zahlen und Gleitkommazahlen

Ob ein Maschinenwort eine Zahl, einen Befehl oder ein Zeichen darstellt, bestimmt allein das Programm. Verdrahtet sind nur Addition und Subtraktion; bei ihnen deutet das Rechenwerk das Wort als **Dualzahl in Festkommadarstellung**. z_0 ist das Vorzeichen, z_{32} das niederwertigste Bit (z_1 ist das höchstwertige Betragsbit); bei vorzeichenlosen Zahlen ist z_0 das höchstwertige Bit. Die Stellung des Kommas wird nicht festgelegt. Multiplikation und Division sind nicht verdrahtet, sondern werden durch Programme mit Gruppenbefehlen gebildet; die Stellung des Kommas legt dort die Programmierung fest (Programmieranleitung 6.1 gebrochen, 6.2 ganz).

Darstellung	Komma	Zahlenwert	Bereich, Besonderheiten
Gebrochene Festkommazahl	z. B. zwischen z_0 und z_1	positiv: $z = \Sigma\, z_i \cdot 2^{-i}$ (i = 1 … 32); negativ: Zweierkomplement 2 − z, d. h. alle Bits vor dem niedrigstwertigen mit 1 besetzten Bit werden negiert	z_0 = Vorzeichen (0 = +)
Ganze Festkommazahl	hinter z_{32}	positiv: $z = \Sigma\, z_i \cdot 2\text{^}(32-i)$, $z_0 = 0$; negativ: Zweierkomplement $2^{33} - z$, $z_0 = 1$; die Null gilt als positiv	$-2^{32} \le z \le 2^{32} - 1$; die kleinste positive Zahl ist die Eins (Wort `00000000001`). Addition und Subtraktion sind dieselben Befehle, nur die Deutung des Wortes ändert sich (Programmieranleitung 6.2)
Gleitkommazahl	—	$z = m \cdot 2^e$ mit normalisierter Mantisse $0{,}5 \le \lvert m\rvert < 1$ für $z \ne 0$; Aufbau des Wortes siehe unten	reine Programmkonvention: der Rechner kennt keine Gleitkommabefehle, das Programm interpretiert das Maschinenwort (Programmieranleitung 6.3, Abb. 28)

Aufteilung des Maschinenwortes bei Gleitkommazahlen:

Bits	Inhalt	Breite
z_0	Vorzeichen der Mantisse (0 = +, 1 = −), Vorzeichen-Betrags-Darstellung	1 Bit
z_1 – z_6	Maschinenexponent $e' = e + 32 = \Sigma\, e_i \cdot 2^i$ (i = 0 … 5); höchstwertiges Bit $e_5 = z_1$, niederwertigstes $e_0 = z_6$	6 Bit
z_7 – z_{32}	Mantissenbetrag $m_1\, m_2 \ldots m_{26}$ ($m_1 = z_7$, $m_{26} = z_{32}$)	26 Bit

- Die Mantisse steht **nicht** im Zweierkomplement, sondern als Betrag mit getrenntem Vorzeichen z_0. Wegen der Normalisierung ist $m_1 = 1$.
- Exponentenbereich $0 \le e' \le 63$, also $-32 \le e \le 31$; Mantissenbereich $0{,}5 \le \lvert m\rvert \le 1 - 2^{-26}$. Zahlenbereich $2^{-33} \le \lvert z\rvert \le (1 - 2^{-26}) \cdot 2^{31}$ sowie $z = 0$.
- **Null:** eindeutig $m_i = 0$ (i = 1 … 26) und $e' = 0$, das ist das Wort `00000000000` (positive Null). Die Bitkombination „negative Null“ ($z_0 = 1$, Rest 0) ist keine gültige Darstellung.
- **Beispiel:** $z = -16 = -0{,}5 \cdot 2^5$ ist `1 100101 1000…0` (Vorzeichen, Exponent $e' = 37$, Mantissenbetrag), oktal `62600000000`; zum Vergleich ist z = +16 das Wort `22600000000`.

Folgerungen aus dem Aufbau (genutzt in den Beispielprogrammen 6.3.1 bis 6.3.3; Testaufträge `Tests/bsp_6_3_1_*.txt`, `bsp_6_3_2_*.txt`, `bsp_6_3_3_*.txt`):

- **Betrag:** Linksverschiebung wirft z_0 heraus, die anschließende Rechtsverschiebung setzt $z_0 = 0$ ein.
- **Vorzeichenumkehr:** Addition der Konstante `40000000000` (nur z_0 gesetzt) kehrt z_0 um. Bei $x < 0$ entsteht dabei ein Überlauf, der durch einen nachfolgenden Überlaufsprung gelöscht werden muss, sonst hält der Rechner an. Die Null darf nicht umgekehrt werden (sonst „negative Null“); sie wird vorher mit dem Vergleichersprung 3 abgefangen.
- **Betragsvergleich:** Weil der Exponent vor der Mantisse steht, ordnen die Betragswörter ($z_0 = 0$) wie die ganzen Zahlen: $|a| < |b|$ genau dann, wenn das Wort von $|a|$ als ganze Zahl kleiner ist. Der markierte Einzelbefehl genügt.

3.2 Externe Darstellung und Tetraden (Konvention des BIS)

Die folgenden Darstellungen gehören nicht zur Hardware, sondern zum Bausteininterpretationssystem BIS (Ergänzung vom 07.10.2026; Quelle: BIS-Dokumentation, siehe PDF „BIS“). Der Rechner kennt weder Gleitkomma noch Dezimalumwandlung noch Tetraden; das Programm bzw. das BIS interpretiert die Maschinenworte.

- **Externe Gleitkommazahl** (Ein- und Ausgabe): entweder übliche Dezimalschreibweise `(±) a,b` oder $z = \pm\, m \cdot 10^E$ mit $0{,}1 \le m < 1$ (für $z \neq 0$). Der Exponent E wird nicht erhöht, er besteht aus Vorzeichen und Betrag. Form: Vorzeichen, höchstens acht Mantissenziffern, Exponentenvorzeichen, Exponent (die Vorzeichen sind optional).
- **Eingabezahlenbereich:** E darf bei der Eingabe nicht größer als 8 sein, also $|z| \le (1 - 10^{-8}) \cdot 10^8$; die kleinste eingebbare Zahl $\neq 0$ ist 10^{-9}. In Dezimalschreibweise ist die größte Zahl 99 999 999, die kleinste $\neq 0$ ist 0,000000001.
- **Genauigkeit:** 26 Dualstellen der Mantisse entsprechen 7 Dezimalstellen; nur 7 Ziffern lassen sich genau verarbeiten. Die achte Mantissenziffer der externen Form ist nicht mehr exakt darstellbar.
- **Tetraden:** Jede Dezimalziffer steht als vierstellige Dualzahl. Ein Maschinenwort fasst höchstens acht Tetraden t_7 … t_0 (z_0–z_{31}, linksbündig); z_{32} bleibt frei und wird nicht ausgewertet. Extern: höchstens acht Dezimalziffern ohne Vorzeichen, $z \le 99\,999\,999$. Die Tetraden t_7 … t_0 sind nicht mit den Oktal-Triaden t_{10} … t_0 zu verwechseln.
- **Beispiel:** Die externe Zahl 2 8 1 3 6 9 0 5 ergibt das Wort `0010 1000 0001 0011 0110 1001 0000 0101 0`, oktal `12011551012`.
- **Rechnen** mit Tetraden ist nicht vorgesehen: Das BIS enthält nur Programme zur Ein- und Ausgabe. Der Assembler `c82ass` kennt keine Gleitkomma- oder Tetraden-Literale; solche Wörter werden als Oktalzahl mit führender 0 mit `dw` angegeben (z. B. −16: `dw 062600000000`).

4 Maschinenwort und Befehlswort

Das interne Maschinenwort hat 33 Bit. Es kann gleichermaßen eine Zahl, einen Befehl oder Alpha-Zeichen darstellen; die Art der Interpretation bestimmt das Programm, auch verschiedenartige Interpretationen innerhalb eines Wortes sind möglich. Ein Befehl belegt die Bits z_{12} bis z_{32} (21 Bit); die zwölf höheren Bits z_0 bis z_{11} sind „beliebig“.

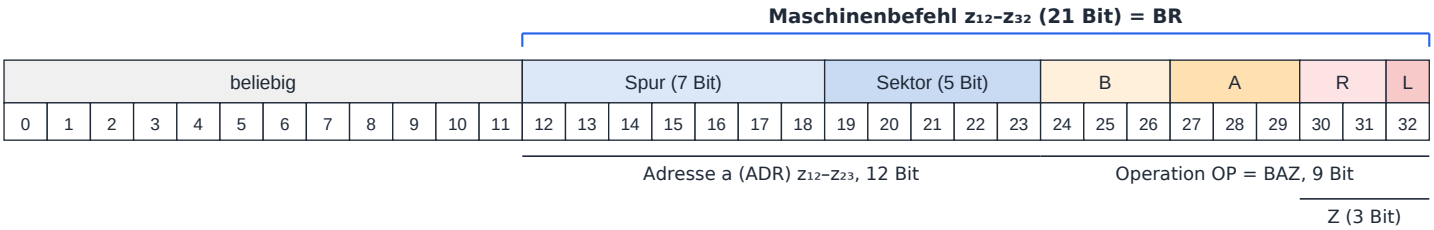


Abbildung 3: Aufbau eines 33-Bit-Maschinenwortes, wenn es als Befehl interpretiert wird. 33 Bit = 11 Oktalziffern (Triaden t_{10} … t_0) · Spur 7 Bit (128 Spuren) · Sektor 5 Bit (32 Sektoren). Bitstelle z_0 … z_{32} (links → rechts) · B = Befehlstyp · A = Operationsart · Z = Operationszusatz = R + L.

Feld	Bits	Breite	Bedeutung
„beliebig“	z_0 – z_{11}	12 Bit	für den Befehl ohne Bedeutung (Original-Bezeichnung)
Maschinenbefehl	z_{12} – z_{32}	21 Bit	steht während der Ausführung im Befehlsregister BR
Adresse a (ADR)	z_{12} – z_{23}	12 Bit	Adreßteil: Spur z_{12}–z_{18} (7 Bit), Sektor z_{19}–z_{23} (5 Bit); in Oktalziffern a_1 a_2 a_3 a_4
Operation (BAZ, OP)	z_{24} – z_{32}	9 Bit	Operationsteil; steht im Operationsregister OR
B – Befehlstyp	z_{24} – z_{26}	3 Bit	Werte 0 … 7, siehe Kapitel 8
A – Operationsart	z_{27} – z_{29}	3 Bit	Bedeutung hängt vom Befehlstyp B ab
Z – Operationszusatz	z_{30} – z_{32}	3 Bit	besteht aus R und L
R – Resultatbehandlung	z_{30} – z_{31}	2 Bit	keine / Rechtsverschiebung / Linksverschiebung / zyklische Linksverschiebung
L – Löschbit	z_{32}	1 Bit	L = 1: der Akkumulator wird in Phase III gelöscht

> **Achtung** Das Feld R (Resultatbehandlung) ist nicht der R-Befehl (indirekter Lesebefehl, Operationscodes mit B ≥ 2, A = 5 und Z ungerade) und auch nicht die Taste bzw. Schalterstellung „R“ (Rechnen) am Bedienpult. Ebenso sind Flag A und Feld A verschieden.

Ein Maschinenbefehl hat die Form ADR OP, z. B. `7774 240`: Adresse 7774, Operation 240 (Transport). Ein Befehl liegt in der Regel als Teil eines Maschinenwortes auf einer Befehlsadresse BADR im Trommelspeicher. Das Operationsregister OR enthält den Operationsteil BAZ, das Befehlsregister BR die Verkettung BR := a ‖ OR.

Adresseinheit (1°): Die Originaldokumentation schreibt 1° für eine Adresseinheit, also die nächste Speicherzelle (z. B. ⟨BZ⟩ + 1° beim Sprungbefehl). In Registern, die nur Adressen enthalten (BZ, AR), und bei Rechnungen mit Adressen ist 1° = 1. Steht eine Adresse im Maschinenwort, so liegt sie in z_{12}–z_{23}, und es folgen die 9 Bit der Operation; in Oktalschreibweise (11 Stellen) lautet das Wort `0000` $a_1a_2a_3a_4$ `BAZ`. Eine im Speicher abgelegte Adresseinheit ist daher im ganzen Maschinenwort nicht 1, sondern 1° = `00000001000` (oktal, kurz `01000`, wie `adreinheit dw 01000` in der Assemblerdokumentation). Der Interpreter rechnet intern mit der Adresse allein (12 Bit), dort ist 1° = 1.

5 Register und Flags

5.1 Register

Register	Breite	Art	Bedeutung
AC	33 Bit	Umlaufregister	Akkumulator; einziger Ausgangspunkt für das Abspeichern in die Trommel
V1 / V2	1 Bit je	Flip-Flops	Vergleicher; zusammen der Pseudovergleicher V3 (V3 = „V1 = 1 und V2 = 1“); dieselben Elemente wie die Flags V1 und V2
OR	9 Bit	statisch (Schiebekette)	Operationsregister, identisch mit BAZ
BZ	12 Bit	Umlaufregister	Befehlszähler; enthält die Adresse BADR des nächstabzuarbeitenden Befehls
AR	12 Bit	statisch (Schiebekette)	Adreßregister; Zellenwähler für jeden Trommelzugriff
BR	21 Bit	—	Befehlsregister, BR := a ‖ OR (Adreßteil a, 12 Bit, und OR, 9 Bit)

BZ und AC sind Umlaufregister: Sie befinden sich technisch auf der Trommel. Ein Lesekopf erfasst die Bit-Belegung eines Platzes, ein Umformer bereitet das erneute Aufsprechen über einen Sprechkopf vor, ein Löschkopf setzt die Plätze auf 0. Die zentrale Taktsteuerung sorgt dafür, dass der Zustand eines Umlaufregisters zu Beginn jeder Wortzeit stets derselbe ist. OR und AR sind statische Register (Schiebekette): Die Information wird seriell ein- und herausgeschoben, der Zustand jeder Stelle ist zu jedem Zeitpunkt abgreifbar.

Zugriffsrichtung zur Trommel: Eine Entnahme gespeicherter Information ist zum Rechenwerk, zum Befehlszähler und zum Operationsregister hin möglich. Eine Abspeicherung ist dagegen nur vom Akkumulator her möglich. Weil es kein Indexregister gibt, laufen Adreßrechnungen über den AC.

5.2 Flags

Ein Flag ist ein elektronischer Speicher für 0 oder 1. Es setzt seinen Wert nicht selbst, sondern wird vom Rechenwerk oder vom Menschen über einen Schalter belegt; wer zuletzt setzt, bestimmt den Zustand. Den meisten Flags ist im Bedienpult eine Lampe zugeordnet. Schalter, Flag und Lampe desselben Namens sind eine Sache: der Schalter ist die Handlung, das Flag der gespeicherte Zustand, die Lampe seine Anzeige.

Flag	Bedeutung
E	= 1, wenn ein Eingabebefehl aufgestellt ist. Zur Eingabe ist zusätzlich die Anwahl eines Eingabegerätes erforderlich. E erlischt nach der Eingabe eines Zeichens mit der Umstellung des Eingabebefehls; E erlischt nicht, wenn der Eingabebefehl nicht ausgeführt wird (z. B. fehlende Anwahl), und leuchtet bei erneutem Eingabebefehl sofort wieder
A	= 1, wenn ein Ausgabebefehl aufgestellt ist. Zur Ausgabe ist zusätzlich die Anwahl eines Ausgabegerätes erforderlich. A erlischt mit der Umstellung des Ausgabebefehls (nach der Übergabe des Zeichens an den 1-Zeichen-Puffer, nicht erst nach Abschluss der Geräteausgabe) und leuchtet bei erneutem Ausgabebefehl sofort wieder. Fehlt die benötigte Anwahl oder ist die programmierte Anwahl nicht existent, stoppt der Rechner mit gesetztem Flag A (Lampe A)
T	Subflag: = 1 für „Tastatur aktiv“ im Kanal K0
SW	Subflag: = 1 für „Schreibwerk aktiv“ im Kanal K0 (T und SW sind alternativ)
K1, K2, K3	= 1, wenn der Kanal 1, 2 bzw. 3 angewählt ist
FE	= 1: „Fehler bei Eingabe“; tritt ein, wenn im 8-Kanal-Kode ein Zeichen mit gerader Zahl von Lochungen gelesen werden soll (Paritätskontrolle). Ausgenommen sind „Irrung Zeichen“ und das Transportloch im Vorspann: Sie werden bei der Lochbandeingabe übersprungen (Originaldokumentation 4.3.2.2, Abb. 16)
FÜ	= 1, wenn ein Überlauf auftritt. Ein Halt wegen Überlauf setzt weder US noch BS; angezeigt wird nur FÜ (Lampe FÜ)
BS	= 1, wenn ein programmierter „bedingter Stopp“ wirksam wird, d. h. wenn der Befehl abgearbeitet wird und der Auswahlschalter BS eingerastet ist. Ist der Schalter nicht eingerastet, bleibt BS = 0 und der Befehl hat keine Wirkung
US	= 1, wenn der Rechner einen „unbedingten Stopp“-Befehl ausführt (immer, sobald der Befehl abgearbeitet wird; der Adreßteil des Befehls kann dem Adreßregister entnommen werden)
V1, V2	Vergleicher; V3 ist die Sprechweise für „V1 = 1 und V2 = 1“
H	= 1, wenn der Schalter H-R auf „H“ (Halt) steht, = 0 bei „R“ (Rechnen)
SP	= 1, wenn der Selektor-Schalter SP gesetzt ist (Bedingung für den bedingten Sprung mit Rückkehrabsicht)

5.3 Anzeige des Arbeitszustands

Die Lampen H, R, US, BS, FÜ, FE, E und A zeigen das Rechnen oder den Grund für die Unterbrechung des Befehlsablaufs an. Ein Anhalten erfolgt grundsätzlich nach der Phase III. Bei gesetztem Schalter H leuchtet die Lampe H; im Zustand „Rechnen“ darf von diesen Lampen nur R leuchten.

- US leuchtet beim Befehl „Unbedingter Stopp“, BS beim programmierten „Bedingten Stopp“ bei gesetztem Schalter BS.
- FÜ leuchtet bei Überlauf (Unterbrechung nur bei Einzelbefehlen ohne Verschiebung, wenn der nachfolgende Befehl kein Überlaufsprung ist).
- FE leuchtet bei einem durch die Paritätskontrolle ermittelten Eingabefehler im 8-Kanal-Kode.
- Die Lampen K0 („T-SW“), K1, K2, K3 zeigen den angewählten Kanal; bei K0 unterscheiden T und SW zwischen Tastatur und Schreibwerk.
- Die Lampen V(1-2) zeigen den Vergleicherstand zur Zeit des Setzens: Lampe 1 = Vergleicher 1, Lampe 2 = Vergleicher 2, beide = Vergleicher 3.
- Die untere Lampenreihe zeigt das Befehlsregister in Triaden: vier Triaden Adreßteil (Gewichte 1000, 100, 10, 1 oktal), drei Triaden Operationsteil (B, A, Z).

6 Die Magnettrommel

Der gesamte Speicher liegt magnetisch auf einer Trommel, organisiert in Zellen (Maschinenworten). Die Trommel hat 128 Spuren (auch Bahnen genannt) mit je 32 Sektoren: 128 × 32 = 4096 Maschinenworte à 33 Bit. Eine 12-Bit-Adresse gliedert sich in die Bahn-Nummer (7 Bit) und die Sektor-Nummer (5 Bit). Die Trommel dreht mit 18 000 min^{-1} (300 Umdrehungen je Sekunde); je Spur werden 32 Wörter à 33 Bit geschrieben, die Arbeitsfrequenz der seriellen Verarbeitung beträgt rund 316 kHz.

Eine Wortzeit ist die Zeit, in der ein gespeichertes Wort unter dem Lesekopf durchläuft: 33 : 316 kHz ≈ 104 µs ≈ 0,1 ms. Die mittlere Zugriffszeit beträgt 1,67 ms, die maximale 3,33 ms.

6.1 Logische und physische Reihenfolge der Sektoren

Die logische Reihenfolge der Sektoren einer Spur (Adressierung) entspricht nicht der physischen Abfolge auf der Trommel. Die Leseelektronik konnte aus Leistungsgründen nicht jeden physisch aufeinanderfolgenden Sektor nacheinander lesen, sondern jeweils nur jeden 6. (gezählt einschließlich des gerade gelesenen, also 5 Positionen weiter; vier Sektoren werden übersprungen). Die logische Sektoradresse wird entsprechend dieser Zugriffsreihenfolge vergeben. Daraus ergibt sich (mindestens teilweise) die Wartezeit.

	00	01	02	03	04	05	06	07
Pos. 00–07	**00**	**15**	**32**	**07**	**24**	**01**	**16**	**33**
	10	11	12	13	14	15	16	17
Pos. 10–17	**10**	**25**	**02**	**17**	**34**	**11**	**26**	**03**
	20	21	22	23	24	25	26	27
Pos. 20–27	**20**	**35**	**12**	**27**	**04**	**21**	**36**	**13**
	30	31	32	33	34	35	36	37
Pos. 30–37	**30**	**05**	**22**	**37**	**14**	**31**	**06**	**23**

Abbildung 4: Physische Zugriffsreihenfolge der 32 Sektoren einer Spur (oktal). Grau: physische Position; in den Kästchen: logische Sektoradresse. Orange: die logischen Sektoren 00–03, die physisch jeweils 5 Positionen auseinander liegen (Schritt +05). Nach einem Zugriff kann erst wieder die 6. logisch folgende Zelle ohne Verlust gelesen werden.

Position (physisch)	**00**	**01**	**02**	**03**	**04**	**05**	**06**	**07**
logische Sektoradresse	00	15	32	07	24	01	16	33
Position (physisch)	10	11	12	13	14	15	16	17
logische Sektoradresse	10	25	02	17	34	11	26	03
Position (physisch)	20	21	22	23	24	25	26	27
logische Sektoradresse	20	35	12	27	04	21	36	13
Position (physisch)	30	31	32	33	34	35	36	37
logische Sektoradresse	30	05	22	37	14	31	06	23

Von einer logischen Adresse L zur logisch nächsten L+1 beträgt der Abstand der Positionen durchgehend +05 (oktal). Umgekehrt entspricht der Schritt zur physisch benachbarten Position durchgehend +15 (oktal) in der logischen Adresse. Beide Schrittweiten sind zueinander invers modulo 40 (oktal): 05 · 15 ≡ 01.

Bahnwahl: Jede Bahn wird über eine eigene Elektronik gewählt; die Schaltzeit ist für alle Bahnen gleich und vernachlässigbar (0 gesetzt). Ein Bahnwechsel kostet daher keine zusätzliche Wartezeit; ausschlaggebend für die Zugriffszeit ist allein die Sektorposition innerhalb der gewählten Bahn.

6.2 Zeitoptimale Programmierung

Anders als bei einem Speicher mit wahlfreiem, gleichmäßig schnellem Zugriff ist die Zugriffszeit variabel: Direkt nach einem Zugriff ist erst wieder die 6. logisch folgende Zelle ohne Wartezeit erreichbar. Liegen Programm und Daten so verteilt, dass der jeweils nächste Zugriff genau auf diese Distanz fällt, ist der Ablauf zeitoptimal; andernfalls muss unter Umständen eine volle Umdrehung abgewartet werden. Ausgehend von der aktuellen Adresse A liegt die frühestens ohne Wartezeit erreichbare Zelle bei (oktal, mod 40):

Befehlsart	Folgeadresse
Einzelbefehl	A+32 ; A+07
Gruppenbefehl	A+32 ; A+07 ; A+24
Sprung	A+24

> **Hinweis** Diese Formeln sind eine Programmier- und Layout-Hilfe zur Vermeidung von Wartezeit, keine Eigenschaft der Befehlsausführung selbst.

Positionen statt Sektornummern (abgeleitet). Der logische Schritt +1 entspricht +05 Positionen. Ein logischer Abstand d (in Sektoren) entspricht daher 5·d mod 32 Positionen (dezimal), jede Position eine Wortzeit (104 µs):

logischer Abstand zum Befehl (oktal, mod 40)	Positionen	Verwendung
+01	5	nächster Befehl bei fortlaufendem Programm
+32	2	Operand eines Einzelbefehls
+07	3	Operand eines Einzelbefehls
+24	4	Folgebefehl nach einem Sprung

Der Operandensektor eines Befehls in Sektor s ist also (s+32) oder (s+07) mod 40 (oktal); die Spur spielt keine Rolle, da ein Bahnwechsel keine Zeit kostet. Die Befehle selbst legt man fortlaufend ab; optimiert wird die Lage der Operanden und der Sprungziele.

Reihenfolge der Phasen und Wartezeiten. Im zeitlichen Ablauf gehört Phase I (Befehlszähleraufstellung) bereits zum nächsten Befehl: die Reihenfolge je Befehl ist II, III, IV, I. Die beiden Wartezeiten heißen W_1 (zwischen Phase I und der Entnahme des nächsten Befehls in Phase II) und W_2 (zwischen Phase III und Phase IV, bis der Operandensektor erreicht ist).

Mindestablauf (abgeleitet), in Positionen ab der Befehlsentnahme (Phase II = 0):

Befehl	Ablauf	Wortzeiten bis zur nächsten Befehlsentnahme
Einzelbefehl, Operand bei +32	II 0, III 1, IV 2, I 3, W_1 4, nächster Befehl 5	5
Einzelbefehl, Operand bei +07	II 0, III 1, W_2 2, IV 3, I 4, nächster Befehl 5	5
Sprung	II 0, III 1, IV 2, I 3, nächster Befehl 4	4

Das deckt sich mit den Zeitangaben in Kapitel 7 (Einzelbefehle mindestens 5 Wortzeiten, Organisationsbefehle mindestens 4 Wortzeiten).

Beispiel 1: Operand nicht optimal. Der Befehl `7716 211` steht in Zelle 4000 (Sektor 00). Sein Operandensektor 16 ist weder +32 noch +07. Ablauf: II in Sektor 00, III in 15, W_2 über 32, 07, 24 und 01, IV in 16, I in 33, dann W_1 von Sektor 10 bis Sektor 24 (der nächsten Umdrehung), nächster Befehl in Sektor 01. Von Befehl zu Befehl vergehen 37 Wortzeiten (abgeleitet: 4 Positionen W_2, 29 Positionen W_1), das ist eine Trommelumdrehung mehr als nötig.

Beispiel 2: Sprung als Umweg. Lässt sich der Operand nicht verlegen, setzt man einen unbedingten Sprung `xx24 000` in die Zelle (Sektor 00), verlegt den Lesebefehl in Sektor 24 und lässt ihn von dort auf den Operandensektor 16 zugreifen (24+32 mod 40 = 16). Der nächste Befehl folgt in Sektor 25. Von Befehl zu Befehl vergehen nur 9 Wortzeiten (Sprung 4, Einzelbefehl 5, abgeleitet). Die Programmieranleitung nennt eine Ersparnis von 28 Wortzeiten (37 − 9 = 28). Der Preis ist eine zusätzliche Zelle für den Sprung; der Befehl in Sektor 24 muss dafür frei sein.

Beispiel 3: Algebraischer Ausdruck (Programmieranleitung 5.19.1, Aufgabe wie 5.5). Transkription nach Seitenabbild; die Spalte „Abstand“ ist abgeleitet (Positionen von der Befehlszelle zum Operanden- bzw. Sprungziel-Sektor, Tabelle in 6.1):

Adr.	Inhalt	Bemerkung	Abstand
3003	3006 000	Sprung zum Programmstart	15
3006	3000 215	⟨AC⟩ := 2a	2
3007	3001 220	⟨AC⟩ := 2a − b	2
3010	3002 224	⟨AC⟩ := 2·(2a − b − c)	2
3011	3031 000	Sprung	16
3031	3000 214	⟨AC⟩ := 2·2·(2a − b − c) + a	3
3032	3001 220	⟨AC⟩ := x	3

Alle Rechenbefehle entnehmen ihren Operanden zeitoptimal (2 bzw. 3 Positionen). Die beiden Sprünge sind dagegen nicht wartefrei (wartefrei wäre +24, also 4 Positionen; von 3011 aus 3035). Das ist offenbar beabsichtigt: Die Sprungziele sind durch die Operandenlage bestimmt, denn Zelle 3000 ist nur von Sektor 06 (+32) oder Sektor 31 (+07) aus wartefrei erreichbar. Nach der Programmieranleitung vergeht von der Entnahme des ersten Befehls bis zur Entnahme des letzten Operanden „genau eine Trommelumdrehung“ weniger als in 5.5 (dort ca. 2⅔ Umdrehungen); diese Angabe ist hier nicht nachgerechnet.

Folgerungen für die Programmierung (abgeleitet).

- Befehle fortlaufend ablegen, Operanden (Variablen, Konstanten, Adreßwörter) auf Sektor +32 oder +07 zum benutzenden Befehl legen. Mit `equ` und `db` lassen sich die dafür nötigen Lücken einstellen; die Lücken können mit Daten gefüllt werden.
- Ein Operand, der von mehreren Befehlen benutzt wird, ist nur für Befehle in passenden Sektoren optimal erreichbar. Sonst entscheidet man je Befehl zwischen Wartezeit und Umweg über einen Sprung.
- Ein Sprung hat sein wartefreies Ziel bei +24. Jedes andere Ziel kostet Wartezeit.

6.3 Bahnsperrenschalter

Die Bahnsperrenschalter schützen innerhalb der Spuren 0000 bis 1740 (oktal) bestimmte, auswählbare Bahnengruppen gegen irrtümliches Überschreiben; der Befehlsablauf bleibt unbeeinflusst (reine Schreibschutz-Funktion). Es gibt sechs Schalter mit den Kennzeichnungen 0001, 0040, 0100, 0200, 0400 und 1000; im Grundzustand sind alle AUS. Der Schalter 0001 („EIN–AUS“) ist der Hauptschalter, die übrigen fünf sind Zusatzschalter. Sie befinden sich physisch an der Rückseite der Tastatur-Anzeige-Einheit; für die Testdefinition gehören sie zum Bedienpult (Bezeichnungen BSP_0001 … BSP_1000).

Bei Hauptschalter AUS sind alle Spuren offen. Bei Hauptschalter EIN gilt eine Bahn B (Bahn-Nr. als Ganzzahl; 0 = Spur 0000, 1 = Spur 0040, … 37 oktal = Spur 1740) genau dann als gesperrt, wenn B eine – auch leere – Bit-Teilmenge der eingeschalteten Zusatzschalter ist. Dabei entsprechen 0040 → Bit 0, 0100 → Bit 1, 0200 → Bit 2, 0400 → Bit 3, 1000 → Bit 4 der Bahn-Nr. Spur 0000 ist damit bei Hauptschalter EIN immer gesperrt. Bahnen ab Spur 2000 (Bahn-Nr. ≥ 32) sind über diese Schalter nie sperrbar.

6.4 Nachbildung im Interpreter: die persistente Trommeldatei

Der Python-Interpreter bildet den Speicher als dauerhafte Textdatei nach (Ergänzung vom 07.10.2026; Vorgabe `Trommel/trommel.txt`, Schalter `--trommel`):

- **Format:** genau 4096 Zeilen `aaaa: tttttttttt` (Adresse und Wert oktal), je Zelle eine Zeile, geordnet nach der **logischen** Adresse `0000` bis `7777` – nicht nach der physischen Reihenfolge der Sektoren aus 6.1.
- **Zeitverhalten:** Wortzeit, Wartezeit und Zugriffszeit werden nicht nachgebildet, alle Zeiten sind 0; die Formeln aus 6.2 bleiben eine Layout-Hilfe.
- **Persistenz:** Der Inhalt bleibt über Tests und Aufrufe hinweg erhalten. Die Zeilen einer Testdatei überlagern ihn, nicht genannte Zellen behalten ihren Wert; jede Zelländerung eines Befehls wird sofort in der Datei nachgeführt. Register, Flags und Geräte beginnen dagegen bei jedem Test neu.
- **Zurücksetzen:** Der Interpreter setzt die Trommel nie zurück und legt die Datei nie an. Eine leere Trommel erzeugt `python3 erzeuge_leere_trommel.py`; fehlt die Datei oder ist sie formal fehlerhaft, startet kein Test (Rückkehrcode 16).

Achtung Ein Test, der Zellen liest oder prüft, die er nicht selbst belegt, hängt vom Trommelstand vor dem Lauf ab und ist nur auf leerer Trommel reproduzierbar. Einzelheiten im PDF „Testhandbuch“ und in „Bedienungsanleitung Python-Interpreter“.

7 Der Befehlszyklus: vier Phasen

Als Zellenwähler wird in jedem Fall das Adreßregister AR benutzt. Bei Entnahmen aus dem Speicher treten wegen der Magnettrommel im Allgemeinen Wartezeiten auf, weil der gewünschte Sektor erst unter die Lese-Schreib-Einrichtung gedreht werden muss. Der Befehlszyklus gliedert sich in vier Phasen; zwischen ihnen liegt jeweils eine Koinzidenz-Wartezeit (K).

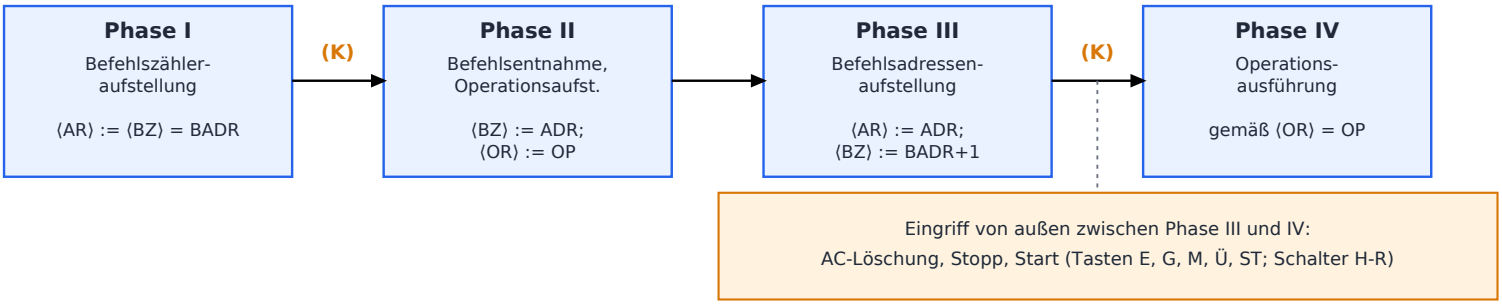


Abbildung 5: Die vier Phasen eines Befehlszyklus. Jede Phase dauert eine Wortzeit (104 µs). (K) = Koinzidenz-Wartezeit: W_1 nach Phase I, W_2 nach Phase III.

Phase	Bezeichnung	Wirkung
I	Befehlszähleraufstellung	Füllen des Zellenwählers AR mit BADR: ⟨AR⟩ := ⟨BZ⟩ = BADR
(K)	W_1 – Wartezeit	Die Trommel dreht sich bis zum Sektor, der durch ⟨AR⟩ = BADR vorgegeben ist.
II	Befehlsentnahme und Operationsaufstellung	Entnahme des Befehls ADR OP: ⟨BZ⟩ := ADR; ⟨OR⟩ := OP
III	Befehlsadressenaufstellung	Füllen von AR mit ADR; gleichzeitig Rücktransport von BADR nach BZ und Erhöhung um 1: ⟨AR⟩ := ⟨BZ⟩ = ADR, gleichzeitig ⟨BZ⟩ := ⟨AR⟩ + 1 = BADR + 1
(K)	W_2 – Wartezeit	Die Trommel dreht sich bis zum Sektor, der durch ⟨AR⟩ = ADR vorgegeben ist.
IV	Operationsausführung	Ausführung entsprechend ⟨OR⟩ = OP

Achtung „Gleichzeitig“ heißt: Beide Teilanweisungen werden mit den Registerwerten ausgewertet, die unmittelbar vor dem Schritt galten. In Phase III ergibt sich das neue BZ deshalb aus dem alten AR (BADR), also BADR + 1, nicht ADR + 1.

Die Tabelle ist ein vergröberter Prinzip-Ablauf: Sie nennt je Phase nur die charakteristische Hauptwirkung. Nebeneffekte, die in der Elektronik gleichzeitig ablaufen, können davon abweichend an anderer Stelle im Zyklus auftreten – das ist keine Inkonsistenz, sondern Folge der vereinfachten Darstellung.

Eingriff von außen: Zwischen Phase III und Phase IV kann von außen eingegriffen werden (AC-Löschung, Stopp, Start). Die Wirkung der Tasten E, G, M, Ü, ST und des Schalters H-R steht im PDF „Bedienpult und Peripherie“.

Zeitbedarf je Befehl (einschließlich Befehlszähler, Befehlsentnahme, Entschlüsselung, Aufstellen der Befehlsadresse und ggf. Wartezeiten):

Befehlsart	minimale Zeit	maximal Op/s
Einzelbefehle	0,5 ms (genauer 5 Wortzeiten = 0,52 ms)	1900
Organisationsbefehle	0,4 ms (genauer 4 Wortzeiten = 0,416 ms)	2400
Gruppen-/Wiederholungsbefehle	—	bis ca. 9000

7.1 Informationszustand nach den Phasen

Die Programmieranleitung (Abschnitt 2.2 „Befehlsschleife“) gibt für jede Phase den Zustand von BZ, AR, OR und AC an. Bezeichnungen: b = Adresse des abzuarbeitenden Befehls, a = sein Adreßteil, ω = sein Operationsteil, ω* = Operationsteil nach einer eventuellen Umstellung in Phase II (R-Befehle, bedingte Sprünge mit erfüllter Bedingung), R_{-1} = Resultat des vorangegangenen Befehls, R = Resultat dieses Befehls. Vor Phase I steht im BZ die Adresse b des nächsten Befehls.

nach Phase	BZ	AR	OR	AC
I	in der Quelle nicht angegeben	b	ω_{-1}	R_{-1}
II	a	b	ω	R_{-1}
III	b+1	a	ω*	in der Quelle nicht angegeben (bei L=1 gelöscht)
IV	b+1	a	ω*	R

- **Wartezeiten:** W_1 liegt zwischen Phase I und Phase II (bis die Sektornummer von b unter dem Kopf steht), W_2 zwischen Phase III und Phase IV (Sektornummer von a). Beide sind Vielfache der Wortzeit. Im Knotengraphen sind das die Koinzidenztests an den Knoten 5/6 (W_1) sowie 26/27 und 57/64 (W_2).
- **Phase IV** kann mehrere Wortzeiten benötigen: je nach Befehlstyp wird die Operation 1- bis 32-mal ausgeführt, jede Ausführung kostet eine Wortzeit.
- **Entfall von W_1:** Endet ein GX- oder WX-Befehl unter den Bedingungen des X-Abbruchs, entfällt die Koinzidenzprüfung vor Phase II (Knoten 4, ANWEISUNG-002).
- **Befehlsentnahme nach X-Abbruch** (Ergänzung 04.10.2026): Ohne Wartezeit W_1 kommt der nächste Befehl nicht aus Zelle b+1, sondern aus der Spur des Befehlszählers und dem Sektor, der in der zweiten Wortzeit nach dem Abbruch unter dem Kopf liegt (Sektor der Zusatzzelle plus zwei physische Schritte, je +15 oktal; Befehlsliste 4.1.6). AR und BZ bleiben b+1, nach Phase III des entnommenen Befehls ist BZ = b+2. Ein Programm muss den Folgebefehl daher in den fälligen Sektor legen (Beispiel: Oktaldruck, Programmieranleitung 5.17, Test `bsp_5_17_oktaldruck`). Siehe auch 8.4.
- **Phase I und BZ (abgeleitet):** Liest man „gleichzeitig“ wie im Knotengraphen (Registerwerte vor dem Knoten), ergibt Phase I für den BZ den alten AR-Wert + 1 statt b+1. Das ist folgenlos, weil Phase II den BZ neu setzt.

7.2 Besonderheiten im Phasenablauf

Quelle: Programmieranleitung, Abschnitt 2.2.2 bis 2.2.4, 4.2.1 und 4.3.2.10.

- **Phase II:** Bei R-Befehlen wird der Adreßteil im BZ disjunktiv mit den Adreßstellen des AC verknüpft und der Operationsteil umgestellt. Bedingte Sprungbefehle werden bei erfüllter Bedingung in unbedingte umgestellt. Als bedingte Sprungbefehle gelten Negativsprung, Positivsprung, Überlaufsprung, Vergleichersprung 1, 2 und 3 sowie der bedingte Sprung mit Rückkehrabsicht.
- **Phase III:** Eine geforderte AC-Löschung geschieht hier. Bei einem Sprung mit Rückkehrabsicht wird der AC disjunktiv mit dem um 1 Bit rechtsverschobenen BZ verknüpft. Ein programmierter Stopp erfolgt am Ende der Phase III, die Fortsetzung hängt von der Bedienung ab. Eingabe-Halt- und Ausgabe-Halt-Befehle werden einer Sonderbehandlung unterworfen (die Programmieranleitung nennt dazu keine Einzelheiten). Zum Ausgabe-Halt gibt die Originaldokumentation (4.3.2.5, Abb. 21) an: Von der Stelle unmittelbar nach der Löschanweisung bis zum Beginn der Phase I vergehen mindestens zwei Wortzeiten; eine Umstellung der Operation nach Phase III in Überlaufsprung oder Vergleichersprung 3 ist bedeutungslos, weil die Sprungbedingung nur in Phase II ausgewertet werden kann (siehe 8.2, Ausgabe-/Anwahlbefehl).
- **Nach Phase III:** Bei einem bedingten Sprung mit nicht erfüllter Bedingung folgt nach einer Wortzeit Phase I. Bei unbedingten Sprüngen und bedingten Sprüngen mit erfüllter Bedingung folgt die Resultatbehandlung in Phase IV, ebenso bei allen anderen Befehlen.
- **Resultatbehandlung:** Sie geschieht nach der Operation in Phase IV. Bei Organisationsbefehlen, die diesen Teil der Befehlsschleife nicht durchlaufen, ist sie wirkungslos, auch wenn die Z-Bits R≠0 enthalten (z. B. Leer-Befehl 164).
- **Markierte Einzelbefehle** erzeugen zusätzlich einen von drei Vergleichszuständen.
- **Überlauf:** Überlauf bedeutet, dass der Übertrag von z_1 nach z_0 vom Übertrag von z_0 zur nicht mehr vorhandenen Maschinenstelle verschieden ist. Er wird nur bei Addition, Subtraktion und bedingter Addition bzw. Subtraktion als Einzelbefehl oder als markierter Einzelbefehl ohne Resultatbehandlung (R=0, d. h. Z=0 oder 1) eingeschaltet, sowie bei Befehlen, die in solche umgestellt werden. Ist im vorhergehenden Befehl ein Überlauf erfolgt und der

abzuarbeitende Befehl kein Überlaufsprung, wird die Befehlsschleife vor Phase IV unterbrochen (im Knotengraphen: Halt wegen Überlauf, Lampe FÜ). Der Überlaufsprung wird dann bereits in Phase II in einen unbedingten Sprung umgestellt, gleichzeitig wird der Überlauf gelöscht.

- **Überlauftest nach Ausgabe-Halt und Anwahl:** Bei vorhandenem Überlauf folgt nach der Zeichenausgabe bzw. der Anwahl ein Halt wegen Überlauf (Knoten 80/81, Ergänzung 04.10.2026); Phase IV läuft bei diesen Befehlen nicht ab, deshalb gibt es keine Resultatbehandlung.

8 Befehlsarten im Überblick

Der Operationsteil BAZ wird bitweise dekodiert; die drei Triaden B, A, Z haben im Sinne eines analytischen Befehlscodes weitgehend selbständige Bedeutung. Wegen des halbanalytischen Aufbaus ist nicht jede der 512 Kombinationen sinnvoll belegt, es gibt auch Kombinationen gleicher Wirkung; daher bleiben etwa 370 verschiedene Befehle. Die vollständige Liste steht im PDF „Einzelbefehlsliste“.

8.1 Befehlstyp B

B	Befehlstyp	Kurzbeschreibung
0	Organisationsbefehl 1. Art	Sprünge, Stopp, Ein-/Ausgabe, Anwahl; je nach Operationsart können die Phasen I oder IV entfallen
1	Organisationsbefehl 2. Art	wie B = 0, Phasenablauf modifizierbar; Vergleichersprünge
2	Einzelbefehl	Operation bezieht sich auf eine Speicherzelle
3	Markierter Einzelbefehl	wie Einzelbefehl, zusätzlich wird ein Vergleichszustand (V1/V2) erzeugt
4	Gruppenbefehl (G)	Phase IV bezieht sich auf alle 32 Zellen der Spur, zu der die Adresse gehört (Phase IV wird 32-mal durchlaufen)
5	Verkürzter Gruppenbefehl (GX)	wie G, zusätzlich Abbruch bei negativem AC (X-Bedingung)
6	Wiederholungsbefehl (W)	wie G, aber Abbruch bei Sektor 23 (oktal); $1 \le n \le 32$ Wiederholungen; Startadresse mit Sektor 0 wirkt wie G
7	Verkürzter Wiederholungsbefehl (WX)	wie W, zusätzlich X-Bedingung

8.2 Operationsart A bei Organisations- und Einzelbefehlen

A	B = 0	B = 1	B = 2 (Einzelbefehl)
0	Unbedingter Sprung	Unbedingter Sprung 2. Art	Konjunktion in AC: ⟨AC⟩ := ⟨AC⟩ ∧ ⟨a⟩
1	Eingabebefehl 1. Art (Eingabe-Halt)	Eingabebefehl 2. Art	Festkomma-Addition: ⟨AC⟩ := ⟨AC⟩ + ⟨a⟩
2	Z = 0–3: Unbedingter Stop · Z = 4–7: Sprung mit Rückkehrabsicht	wie B = 0 (2. Art)	Festkomma-Subtraktion: ⟨AC⟩ := ⟨AC⟩ − ⟨a⟩
3	Ausgabe-/Anwahlbefehl	Ausgabe-/Anwahlbefehl	bedingte Addition/Subtraktion
4	Negativ-Sprung (⟨AC⟩ < 0)	Vergleicher-Sprung 1 (V1 oder V3)	Transport: ⟨a⟩ := ⟨AC⟩
5	Positiv-Sprung (⟨AC⟩ ≥ 0)	Vergleicher-Sprung 2 (V2 oder V3)	Z gerade: Konjunktion im Speicher · Z ungerade: R-Befehl (indirektes Lesen)
6	Z = 0–3: Bedingter Stop (Schalter BS) · Z = 4–7: Bedingter Sprung mit Rückkehrabsicht (Schalter SP)	Z = 0–3: Bedingter Stop 2. Art · Z = 4–7: Leer-Befehl	Konjunktion im Speicher mit negiertem AC
7	Überlauf-Sprung	Vergleicher-Sprung 3 (nur V3)	Vorzeichenbefehl (bedingte Speicherlöschung) bzw. R-Minuslesebefehl

Hinweis Varianten bei B = 2 (Befehlsliste 4.3; gleiches gilt für die Typen 3 bis 7): **A = 3, Z = 2 oder 3:** ist das niederwertigste Bit z_{32} des AC gleich 1, wird ⟨a⟩ addiert, dann folgt die Rechtsverschiebung (Anwendung: Multiplikation, Programmieranleitung 6.1.2 und 6.2.1); z_{32} ist hier **nicht** das Löschbit. **A = 7, Z = 2:** Speicherlöschung, wenn z_{32}(AC) = 0. **A = 7, Z = 3:** ⟨AC⟩ := 0.

Notation der Sprungbefehle (Originaldokumentation): Jeder Sprungbefehl ist eine Fallunterscheidung: nächster Befehl := a, falls die Sprungbedingung erfüllt ist, sonst nächster Befehl := ⟨BZ⟩ + 1°. Beim unbedingten Sprung

(000) gilt immer a. Die Bedingungen: 040 ⟨AC⟩ < 0; 050 ⟨AC⟩ ≥ 0; 070 Überlauf; 140 Vergleicher 1 oder 3; 150 Vergleicher 2 oder 3; 170 Vergleicher 3 (gilt für alle Z-Werte). 1° ist die Adresseinheit (Kapitel 4). Bei nicht erfüllter Bedingung wird der Befehl aus Zelle b+1 geholt, ohne Phase IV und ohne Verschiebung.

Ausgabe-/Anwahlbefehl (030–037): Die Anwahl erfolgt mit a = 4700 (Kanal 0: Tastatur oder SW-Eingabe), 4600 (Kanal 1: Leser 1 oder Stanzer) und 4500 (Kanal 2: Leser 2 oder SW-Ausgabe); sie bleibt bis zur nächsten Anwahl erhalten. Ist $z_{12} = 0$, wird ein Zeichen ausgegeben (Stanzer: Bits z_{13}–z_{20}, 1 = gelocht; Schreibwerk: nur z_{14}, z_{15}, z_{17}–z_{20}). Fehlt das Ausgabegerät (K0, K3, keine Anwahl) oder wird eine nicht existierende Anwahl programmiert, stoppt der Rechner mit gesetztem Flag A. Der AC unterliegt nur der Löschanweisung; Phase IV läuft nicht ab. Die acht Operationsteile 030–037 unterscheiden sich nur durch die Löschanweisung. Von der Löschanweisung (Phase III) bis zum Beginn der Phase I vergehen mindestens zwei Wortzeiten (Originaldokumentation 4.3.2.5, Abb. 21; im Interpreter Wartezeit 0). Eine Umstellung der Operation nach Phase III in Überlaufsprung oder Vergleichersprung 3 ist für die Wirkung bedeutungslos, weil die Sprungbedingung nur vorher in Phase II ausgewertet wird. Bei vorhandenem Überlauf folgt nach der Ausgabe bzw. Anwahl ein Halt wegen Überlauf.

Markierte Einzelbefehle (B = 3): Zur Wirkung des Einzelbefehls kommt das Setzen des Vergleichers hinzu. Verglichen wird der Akkumulatorinhalt nach Phase III (bei L = 1 also 0) mit v; dabei ist v = ⟨a⟩ für A ≤ 3 und v = 0 für A ≥ 4. Am Ende von Phase III werden beide Vergleicher eingeschaltet (V3); in Phase IV wird nur bei Ungleichheit korrigiert, und zwar stets entgegengesetzt: Bei ⟨AC⟩ < v bleibt V1 an, bei ⟨AC⟩ > v bleibt V2 an, bei Gleichheit bleiben beide an. Bei Gruppen- und Wiederholungsbefehlen (B = 4 … 7) gilt dieselbe Operationsart wie bei B = 2, ausgeführt für mehrere Zellen.

8.3 Operationszusatz Z

Z (oktal)	Löschbit L	R (Wert)	Wirkung
0	0	0	keine Löschung, keine Resultatbehandlung
1	1	0	Löschen des AC (in Phase III), keine Resultatbehandlung
2	0	1	keine Löschung, Rechtsverschiebung: ⟨AC⟩ := ½·⟨AC⟩, z_0 := 0
3	1	1	Löschen, Rechtsverschiebung
4	0	2	keine Löschung, Linksverschiebung: ⟨AC⟩ := 2·⟨AC⟩, z_{32} := 0
5	1	2	Löschen, Linksverschiebung
6	0	3	keine Löschung, zyklische Linksverschiebung (z_0 wird nach z_{32} übertragen)
7	1	3	Löschen, zyklische Linksverschiebung

Die Resultatbehandlung geschieht nach der Operation in Phase IV und bezieht sich auf den Akkumulator; bei einigen Organisationsbefehlen wird dieser Teil nicht durchlaufen und ist dann wirkungslos. Das Löschbit wirkt in Phase III. Kein Verschiebebefehl erzeugt einen Überlauf: ein Überlauf entsteht nur bei Addition, Subtraktion, bedingter Addition/Subtraktion oder R-Operation, und nur bei R = 0 (Z = 0 oder 1, ohne Resultatbehandlung; Programmieranleitung 4.3.2.10).

8.4 Abbruch bei Gruppen- und Wiederholungsbefehlen

- **Normal (G, GX):** nach 32 bearbeiteten Sektoren. **W, WX:** mit der Zelle des Sektors 23 (bei Startadresse mit Sektor 0 nach 32 Sektoren). Sektor 23 liegt in der Tabelle in 6.1 auf Position 37, ist also die physisch letzte Zelle der Spur (abgeleitet); die Befehlsliste formuliert dasselbe als „Abbruch, sobald die Sektornummer 00 erreicht ist“.
- **X-Bedingung (GX, WX):** Der Abbruch erfolgt, wenn der AC negativ wird; Vorzeichen ist z_0, bei Rechtsverschiebung (R = 1) stattdessen z_{32}. Operation und Resultatbehandlung werden noch mit einer weiteren Zelle (Zusatzzelle) ausgeführt, danach folgt der Abbruch.
- Erfüllen sich normale und X-Bedingung gleichzeitig, hat die normale Bedingung Vorrang (Abb. 10 der Originaldokumentation, Befehlsliste 4.1.6).
- Nach einem X-Abbruch wird der nächste Befehl ohne Wartezeit W_1 sofort entnommen: Die Spur ergibt sich aus dem Befehlszähler, der Sektor aus der Trommellage in der zweiten Wortzeit nach dem Abbruch (in der ersten folgt Phase I). Das ist der Sektor der Zusatzzelle plus zwei physische Schritte (je +15 oktal) und im Allgemeinen nicht der Sektor von b+1. Wurde der X-Befehl aus der Zelle b entnommen, so hat der Befehlszähler nach Phase III des nächsten Befehls den Inhalt b+2; ist der nächste Befehl ein tatsächlich ausgeführter Sprung, wird dieser Inhalt nicht ausgewertet, sonst kommt der folgende Befehl aus Zelle b+2.

> **Achtung** Auch im Interpreter, der sonst ohne Zeiten rechnet, hängt das Ergebnis nach einem X-Abbruch von der Trommellage ab: Der Folgebefehl muss im fälligen Sektor liegen, nicht in Zelle b+1 (Kapitel 9).

9 Hardware und interpretative Abarbeitung

Das Gesamtprojekt beschreibt zwei Aspekte: erstens die Hardware mit den in der Elektronik angelegten Beziehungen, die als Knotenautomat rekonstruiert wurde, und zweitens davon abweichend die Annahmen für eine interpretative Abarbeitung eines Befehls. Der Knotenautomat hat 80 Knoten (Nummern 1 bis 81, Nr. 77 entfällt): 37 Anweisungen, 27 Tests, 10 Konnektoren und 6 Phasenzuordnungen für die vier Phasen. Wichtigste Annahme der interpretativen Abarbeitung: Alle Zeiten sind 0. Zeitliche Abhängigkeiten werden in ihrer Reihenfolge betrachtet, führen aber nicht zu Wartezeiten; die Trommeldrehung wird nicht nachgebildet. Die Sektorfolge der Trommel geht dennoch ein: Gruppen- und Wiederholungsbefehle bearbeiten die Zellen in der physischen Folge (ANWEISUNG-025: a := a+15 mod 40), und nach einem X-Abbruch bestimmt die Trommellage den Sektor der Befehlsentnahme (8.4).

Der Speicher ist im Interpreter die persistente Trommeldatei (6.4). Der Vollständigkeitstest `c8205_befehlssweep.py` prüft alle 512 Operationscodes gegen den Knotengraphen; einen Stub gibt es für keinen Code mehr, nur K3 wird nicht benutzt.

Die Phasen I bis IV entsprechen den Phasenzuordnungs-Knoten des Knotengraphen (Beginn Phase I: Knoten 2, Phase II: Knoten 8, Phase III: Knoten 15, Phase IV: Knoten 31, 61 und 67). Der Knotengraph wird im PDF „Knotengraph“ dargestellt; wie man ihn ablaufen lässt, steht im „Testhandbuch“.

Stopp-Zustände im Knotengraphen: Ein programmierter Stopp erfolgt am Ende der Phase III. Der unbedingte Stopp (Knoten 45) setzt nur US, der bedingte Stopp bei gesetztem Schalter BS (Knoten 79) nur BS. Ein Halt wegen Überlauf (Knoten 30, 46, 58, 66 und 81) setzt weder US noch BS; angezeigt wird nur FÜ. Die Fortsetzung nach einem Stopp hängt von der Bedienung ab und ist im Knotenautomaten nicht durch eigene Knoten abgebildet (Endzustände ohne Ausgang).

Einstiegsknoten nach einem Stopp: Der Interpreter wertet die Tastenfolge aus, stellt das Befehlsregister auf und beginnt bei einem Einstiegsknoten (Originaldokumentation 4.3.2.3, Abb. 19; Einzelheiten im PDF „Bedienpult und Peripherie“):

Tastenfolge	Einstiegsknoten	Folge
ST, Ü, E ohne M	19	Ausführung des aufgestellten BR, z. B. bei B = 2: Wartezeit W_2, Überlauftest, Phase IV
M ohne E, BR = Sprung mit Rückkehrabsicht (025, 027, 125, 127)	54 (Konnektor 4)	Phase III gilt als abgelaufen; nur Überlauftest und Resultatbehandlung, dann Phase II
M ohne E, anderer BR	19	Ausführung des BR ab Knoten 19, z. B. bedingter Sprung mit Rückkehrabsicht 065/067: Knoten 24, 55, 60, dann Phase I

> **Achtung** Die Zeitangaben aus Kapitel 6 und 7 (104 µs, Zugriffszeiten, Op/s) beschreiben den realen Rechner. Der Interpreter ermittelt weder Wortzeiten noch Wartezeiten; seine „Knotenzahl“ im Protokoll ist keine Zeitangabe. Um die zeitoptimale Programmierung (6.2) am Interpreter zu prüfen, müsste dieser die Trommelposition (Wortzeitzähler mod 32 und Reihenfolge der Magnettrommel-Tabelle) mitführen.

10 Wo steht was? – Verweise

Thema	Dokument
Tasten, Schalter, Lampen, Kanalanwahl, Geräte	PDF „Bedienpult und Peripherie“
Alle 512 Operationscodes mit Wirkung	PDF „Einzelbefehlsliste“
Zeichenkodes (Lochstreifenkode, R 300, CCITT-2)	PDF „Kodetabellen“
Ablauf der Befehlsverarbeitung als Graph	PDF „Knotengraph“ (Tabellenform: „Knotentabellen“)
Tests schreiben und Protokolle lesen; Trommel bei Tests	PDF „Testhandbuch“
Bedienung des Python-Interpreters	PDF „Bedienungsanleitung Python-Interpreter“
Bausteininterpretationssystem BIS (Dezimalzahlen, Gleitkomma, Tetraden, Kleines Eingabeprogramm)	PDF „BIS“
Assembler c82ass	PDF „Assembler-Dokumentation“
Projektüberblick, Dokumentenlandkarte	PDF „Projektüberblick“